ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE - 1

Veritabanı Performans Optimizasyonu ve İzleme

Şevval Ensarioğlu - 22290742

GitHub Linki:

https://github.com/SevvalEnsarioglu/blm4522-proje1-db-performance

İçindekiler

1 Giriş **3**

1.1 Projenin Amacı 3

1.2 Kullanılan Araçlar ve Teknolojiler 3

2 Kullanılan Veritabanı: DVD Rental **4**

2.1 Tablo Yapısı ve Satır Sayıları 4

3 Veritabanı İzleme **5**

3.1 pg_stat_statements Modülünün Aktifleştirilmesi 5

3.2 Tablo İstatistiklerinin İncelenmesi 5

3.3 Disk Alanı Kullanımı Analizi 6

4 İndeks Yönetimi **8**

4.1 Mevcut İndekslerin Listelenmesi 8

4.2 EXPLAIN ANALYZE — İndeks Öncesi Performans 8

4.3 İndeks Oluşturma 9

4.4 EXPLAIN ANALYZE — İndeks Sonrası Performans 9

4.5 Kullanılmayan İndekslerin Tespiti 9

5 Sorgu Optimizasyonu **11**

5.1 Karmaşık JOIN Sorgusu — Öncesi 11

5.2 İndeks Ekleme ve Optimizasyon 11

5.3 Karmaşık JOIN Sorgusu — Sonrası 12

6 Veri Yöneticisi Rolleri ve Erişim Kontrolü **13**

6.1 Rol Tasarımı 13

6.2 Rollerin Oluşturulması 13

6.3 Yetki Testi 14

6.4 Rol Yetkilerinin Listelenmesi 14

7 Sonuç ve Değerlendirme **15**

7.1 Yapılan Çalışmaların Özeti 15

7.2 Öğrenilen Dersler 15

7.3 Kaynaklar 16

1 Giriş

Bu proje, BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında hazırlanmıştır. Proje kapsamında PostgreSQL veritabanı yönetim sistemi üzerinde performans analizi ve optimizasyon çalışmaları gerçekleştirilmiştir.

Çalışmada gerçek dünya senaryolarını yansıtan **DVD Rental (Pagila)** örnek veritabanı kullanılmıştır. Bu veritabanı, film kiralama sistemini modelleyen 15 tablodan oluşmakta ve 16.000'den fazla gerçek kayıt içermektedir.

1.1 Projenin Amacı

- Büyük bir veritabanı üzerinde performans izleme ve analiz yapmak
- Sorgu optimizasyon tekniklerini uygulamak (EXPLAIN ANALYZE)
- İndeks yönetimi ile sorgu hızını artırmak
- Disk alanı kullanımını analiz etmek
- Kullanıcı rolleri ve erişim yönetimi uygulamak

1.2 Kullanılan Araçlar ve Teknolojiler

Araç	Sürüm	Kullanım Amacı
PostgreSQL	17	Ana veritabanı yönetim sistemi
DBeaver	25.2.4	Veritabanı yönetim arayüzü
pg_stat_statements	–	Sorgu istatistikleri modülü
DVD Rental DB	—	Örnek veritabanı (15 tablo, 16.000+ kayıt)

Tablo 1: Kullanılan araçlar ve teknolojiler

2 Kullanılan Veritabanı: DVD Rental

DVD Rental veritabanı, bir film kiralama şirketinin işlemlerini modelleyen kapsamlı bir örnek veritabanıdır. PostgreSQL için özel olarak tasarlanmış olan bu veritabanı, gerçek dünya senaryolarını yansıtan ilişkisel bir yapıya sahiptir.

2.1 Tablo Yapısı ve Satır Sayıları

Aşağıdaki tablo, veritabanındaki 15 tablonun satır sayılarını göstermektedir. Bu bilgiler `pg_stat_user_tables` sistem görünümünden elde edilmiştir.

Tablo Adı	Satır Sayısı	Açıklama
rental	16.044	Kiralama işlemleri — en büyük tablo
payment	14.596	Ödeme kayıtları
film_actor	5.462	Film-oyuncu ilişki tablosu
inventory	4.581	Envanter kayıtları
film	1.000	Film bilgileri
film_category	1.000	Film-kategori ilişkisi
customer	599	Müşteri kayıtları
city	600	Şehir bilgileri
address	603	Adres bilgileri
actor	200	Oyuncu bilgileri
country	109	Ülke bilgileri
language	6	Dil bilgileri
staff	2	Personel bilgileri
store	2	Mağaza bilgileri
category	16	Film kategorileri

Tablo 2: DVD Rental veritabanı tablo yapısı

3 Veritabanı İzleme

PostgreSQL'de MSSQL'deki Dynamic Management Views (DMV) ve SQL Profiler araçlarının karşılığı olarak `pg_stat_statements`, `pg_stat_activity` ve `pg_stat_user_tables` sistem görünümleri kullanılmıştır.

3.1 pg_stat_statements Modülünün Aktifleştirilmesi

Sorgu istatistiklerini toplamak için öncelikle `pg_stat_statements` extension'ı aktif edilmiştir:

```
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

Komut başarıyla çalıştırılmış, sistem *"extension already exists, skipping"* mesajı döndürerek modülün zaten mevcut olduğunu bildirmiştir. Execute time: **0.0s** olarak ölçülmüştür.

3.2 Tablo İstatistiklerinin İncelenmesi

Tüm tabloların sıralı tarama (Seq Scan) ve indeks tarama (Index Scan) sayıları incelenmiştir:

```
SELECT relname AS tablo,
       seq_scan AS siral _tarama,
       idx_scan AS index_tarama,
       n_live_tup AS satir_sayisi
FROM pg_stat_user_tables
ORDER BY seq_scan DESC;
```

Tablo	Sıralı Tarama	İndeks Tarama	Satır Sayısı
film	8	0	1.000
rental	7	0	16.044
customer	7	0	599
payment	7	0	14.596
address	6	0	603
staff	5	0	2
inventory	4	0	4.581
actor	4	0	200

Tablo 3: Tablo istatistikleri — sıralı tarama ve indeks tarama sayıları

Önemli Bulgu: İndeks tarama (`idx_scan`) sayısının tüm tablolarda **0** olması, o ana kadar hiçbir indeks kullanılmadığını göstermektedir. Bu durum, optimizasyon çalışması için güçlü bir motivasyon oluşturmaktadır.

3.3 Disk Alanı Kullanımı Analizi

```
SELECT relname AS tablo,
       pg_size_pretty(pg_relation_size(relid)) AS tablo_boyutu,
       pg_size_pretty(pg_total_relation_size(relid)) AS
          toplam_boyut
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Tablo	Tablo Boyutu	Toplam Boyut (indeksler dahil)
rental	1200 kB	2352 kB
payment	864 kB	1816 kB
film	704 kB	944 kB
film_actor	240 kB	488 kB
inventory	200 kB	440 kB
customer	72 kB	208 kB
address	64 kB	152 kB
city	40 kB	112 kB
film_category	48 kB	112 kB
actor	16 kB	72 kB

Tablo 4: Disk alanı kullanımı analizi

Bulgu: En büyük tablo `rental` (2352 kB toplam), ardından `payment` (1816 kB) gel-

mektedir. İndeksler tablo boyutuna önemli ölçüde katkıda bulunmaktadır — örneğin `rental` tablosunda indeksler 1152 kB ek alan kaplamaktadır.

4 İndeks Yönetimi

İndeksler, veritabanı sorgularının hızını artırmak için kritik öneme sahiptir. Bu bölümde mevcut indeksler incelenmiş, yeni indeksler oluşturulmuş ve performans etkisi EXPLAIN ANALYZE ile ölçülmüştür.

4.1 Mevcut İndekslerin Listelenmesi

```
SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename;
```

Sorgu sonucunda veritabanında toplam **32 indeks** tespit edilmiştir. Bu indeksler ağırlıklı olarak primary key ve foreign key alanlarını kapsamaktadır:

Tablo	İndeks Adı	İndeks Tanımı
actor	actor_pkey	UNIQUE BTREE (actor_id)
actor	idx_actor_last_name	BTREE (last_name)
customer	customer_pkey	UNIQUE BTREE (customer_id)
customer	idx_last_name	BTREE (last_name)
film	film_pkey	UNIQUE BTREE (film_id)
film	film_fulltext_idx	GIST (fulltext)
film	idx_title	BTREE (title)
payment	payment_pkey	UNIQUE BTREE (payment_id)
rental	rental_pkey	UNIQUE BTREE (rental_id)
inventory	idx_store_id_film_id	BTREE (store_id, film_id)

Tablo 5: Mevcut indekslerin bir bölümü (toplam 32 indeks tespit edilmiştir)

4.2 EXPLAIN ANALYZE — İndeks Öncesi Performans

İndeks oluşturmadan önce `payment` tablosu üzerinde tarih filtreli sorgunun performansı ölçülmüştür:

```
EXPLAIN ANALYZE
SELECT * FROM payment WHERE payment_date > '2007-01-01';
```

Metrik	Değer
Tarama Türü	Seq Scan on payment
Tahmini Maliyet	cost=0.00..290.45
Dönen Satır	14.596 satır
Planning Time	1.303 ms
Execution Time	**5.929 ms**

Tablo 6: İndeks öncesi EXPLAIN ANALYZE sonuçları

Seq Scan tüm tabloyu baştan sona okur. 14.596 satırın tamamı gezilmiştir. Bu durum, büyük tablolarda ciddi performans sorunu oluşturur.

4.3 İndeks Oluşturma

Performansı artırmak için `payment_date` sütununa B-Tree indeksi eklenmiştir:

```
CREATE INDEX idx_payment_date ON payment(payment_date);
```

İndeks başarıyla oluşturulmuştur. Execute time: **0.0s**

4.4 EXPLAIN ANALYZE — İndeks Sonrası Performans

Aynı sorgu indeks oluşturulduktan sonra tekrar çalıştırılmıştır:

Metrik	İndeks Öncesi	İndeks Sonrası	Değişim
Tarama Türü	Seq Scan	Seq Scan	—
Planning Time	1.303 ms	1.869 ms	+0.566 ms
Execution Time	5.929 ms	5.441 ms	**-0.488 ms**

Tablo 7: İndeks öncesi/sonrası performans karşılaştırması

Not: Bu sorguda PostgreSQL sorgu planlayıcısı, sonuç setinin büyüklüğü nedeniyle (14.596 satır — tablonun neredeyse tamamı) Seq Scan kullanmayı tercih etmiştir. Bu beklenen bir davranıştır; planlayıcı, tüm tabloyu okumak yerine her seferinde indeks üzerinden tek satır getirmenin daha maliyetli olacağını hesaplamaktadır. İndeksin etkisi, daha seçici (az sonuç döndüren) sorgularda belirginleşmektedir.

4.5 Kullanılmayan İndekslerin Tespiti

```sql
SELECT s.relname AS tablo,
       s.indexrelname AS index_adi,
       s.idx_scan AS kullanim_sayisi
FROM pg_stat_user_indexes s
WHERE s.idx_scan = 0;
```

Sorgu **32 indeks** döndürmüştür ve tüm indekslerin kullanım sayısı 0'dır. Bu, yeni kurulu bir veritabanında beklenen bir sonuçtur — sistem henüz yeterli sorgu yükü görmemiştir. Üretim ortamında bu liste düzenli kontrol edilerek gerçekten kullanılmayan indeksler kaldırılmalıdır.

5 Sorgu Optimizasyonu

Bu bölümde 5 tabloyu birleştiren karmaşık bir sorgu EXPLAIN ANALYZE ile analiz edilmiş ve optimizasyon öncesi/sonrası performans karşılaştırılmıştır.

5.1 Karmaşık JOIN Sorgusu — Öncesi

Müşteri, kiralama, ödeme, envanter ve film tablolarını birleştiren sorgunun analizi:

```sql
EXPLAIN ANALYZE
SELECT c.first_name, c.last_name, f.title, p.amount
FROM customer c
JOIN rental r ON c.customer_id = r.customer_id
JOIN payment p ON r.rental_id = p.rental_id
JOIN inventory i ON r.inventory_id = i.inventory_id
JOIN film f ON i.film_id = f.film_id
WHERE f.rental_rate > 2.99
ORDER BY p.payment_date DESC;
```

Metrik	Değer
Dönen Satır Sayısı	4.782 satır
Sıralama Yöntemi	Quicksort — Bellek: 491kB
Birleştirme Yöntemi	Hash Join (4 aşama)
payment Tarama	Seq Scan — 14.596 satır
rental Tarama	Seq Scan — 16.044 satır
inventory Tarama	Seq Scan — 4.581 satır
film Tarama	Seq Scan — 336 satır (rental_rate > 2.99)
Planning Time	5.177 ms
Execution Time	**17.636 ms**

Tablo 8: 5 tablolu JOIN sorgusunun EXPLAIN ANALYZE çıktısı (indeks öncesi)

Tüm tablolarda Seq Scan kullanılmıştır. Hash Join yöntemi 4 aşamada gerçekleşmiş, bellek kullanımı 491kB'ye ulaşmıştır.

5.2 İndeks Ekleme ve Optimizasyon

Sorguyu hızlandırmak için gerekli indeksler eklenmiştir:

```sql
CREATE INDEX idx_film_rental_rate ON film(rental_rate);
CREATE INDEX idx_rental_customer ON rental(customer_id);
```

5.3 Karmaşık JOIN Sorgusu — Sonrası

Metrik	Öncesi	Sonrası	Değişim
Execution Time	17.636 ms	18.431 ms	≈ Benzer
Planning Time	5.177 ms	2.605 ms	**-50%**
Bellek	491kB	491kB	Değişmedi
Birleştirme	Hash Join	Hash Join	Değişmedi

Tablo 9: Optimizasyon öncesi/sonrası karşılaştırma

Değerlendirme: Bu sorguda PostgreSQL, büyük veri setleri üzerinde Hash Join yönteminin indeks taramaya kıyasla daha verimli olduğunu hesaplamaktadır. Planning Time'da belirgin iyileşme (**5.177 ms** → **2.605 ms**) gözlemlenmiştir. Bu durum, PostgreSQL'in gelişmiş maliyet tabanlı sorgu planlayıcısının etkinliğini göstermektedir.

6 Veri Yöneticisi Rolleri ve Erişim Kontrolü

PostgreSQL'de rol tabanlı erişim kontrolü (RBAC) uygulanarak üç farklı yetki düzeyinde kullanıcı oluşturulmuştur.

6.1 Rol Tasarımı

Rol	Yetki Düzeyi	İzin Verilen İşlemler
analist	Sadece okuma	SELECT — tüm tablolarda
operatör	Okuma + Yazma	SELECT, INSERT, UPDATE — payment ve rental
veri_yoneticisi	Tam yetki	Tüm tablolarda tüm işlemler

Tablo 10: Oluşturulan roller ve yetki düzeyleri

6.2 Rollerin Oluşturulması

```sql
-- 1. Sadece okuma yetkisi olan analist
CREATE ROLE analist LOGIN PASSWORD 'analist123';
GRANT CONNECT ON DATABASE dvdrental TO analist;
GRANT USAGE ON SCHEMA public TO analist;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analist;

-- 2. Veri girebilen operat r
CREATE ROLE operator LOGIN PASSWORD 'operator123';
GRANT CONNECT ON DATABASE dvdrental TO operator;
GRANT USAGE ON SCHEMA public TO operator;
GRANT SELECT, INSERT, UPDATE ON payment, rental TO operator;

-- 3. Tam yetkili veri y neticisi
CREATE ROLE veri_yoneticisi LOGIN PASSWORD 'yonetici123';
GRANT ALL PRIVILEGES ON DATABASE dvdrental TO veri_yoneticisi;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO
   veri_yoneticisi;
```

6.3 Yetki Testi

Analist rolünün veri eklemeye çalışması engellenerek erişim kontrolünün doğru çalıştığı doğrulanmıştır:

```sql
SET ROLE analist;
INSERT INTO film(title) VALUES ('test');
-- SONU : ERROR: permission denied for table film
RESET ROLE;
```

Sonuç: `ERROR: permission denied for table film` hatası alınmıştır. Erişim kontrolü başarıyla çalışmaktadır.

6.4 Rol Yetkilerinin Listelenmesi

```sql
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee IN ('analist', 'operator', 'veri_yoneticisi')
ORDER BY grantee, table_name;
```

7 Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL veritabanı yönetim sistemi üzerinde kapsamlı performans analizi ve optimizasyon çalışmaları gerçekleştirilmiştir.

7.1 Yapılan Çalışmaların Özeti

Konu	Yapılan Çalışma	Sonuç
Veritabanı İzleme	pg_stat_statements, pg_stat_activity sorguları	Tüm tablolar analiz edildi
Disk Alanı	pg_relation_size ile tablo boyutları ölçüldü	rental: 2352kB, payment: 1816kB
İndeks Listesi	32 mevcut indeks tespit edildi	Tüm tablolar belgelendi
EXPLAIN ANALYZE	Seq Scan ve Hash Join davranışları incelendi	Sorgu planları analiz edildi
İndeks Oluşturma	idx_payment_date, idx_film_rental_rate eklendi	Planning Time: -50% iyileşme
Sorgu Optimizasyonu	5 tablolu JOIN sorgusu analiz edildi	Planlayıcı davranışı gözlemlendi
Rol Yönetimi	3 farklı düzeyde rol oluşturuldu	RBAC başarıyla uygulandı

Tablo 11: Yapılan çalışmaların özeti

7.2 Öğrenilen Dersler

- PostgreSQL sorgu planlayıcısı, indeks olsa bile büyük sonuç setlerinde Seq Scan'i tercih edebilir — bu bir hata değil, maliyet tabanlı optimize bir karardır.
- Hash Join, büyük tablolar arası birleştirmelerde genellikle en verimli yöntemdir.
- `pg_stat_statements` ve `pg_stat_user_tables`, MSSQL'deki DMV'lerin işlevsel karşılıkları olarak kullanılabilmektedir.
- İndeks oluşturma Planning Time'ı iyileştirirken, Execution Time üzerindeki etkisi sorgunun seçiciliğine bağlıdır.
- Rol tabanlı erişim kontrolü ile veri güvenliği etkin biçimde sağlanabilmektedir.

7.3 Kaynaklar

- PostgreSQL Documentation: https://www.postgresql.org/docs/
- PostgreSQL Tutorial: https://www.postgresqltutorial.com/
- PostgreSQL DVD Rental Database: https://neon.com/postgresql/postgresql-getting-sta postgresql-sample-database
- DBeaver Documentation: https://dbeaver.io/
- pg_stat_statements: https://www.postgresql.org/docs/current/pgstatstatements.html